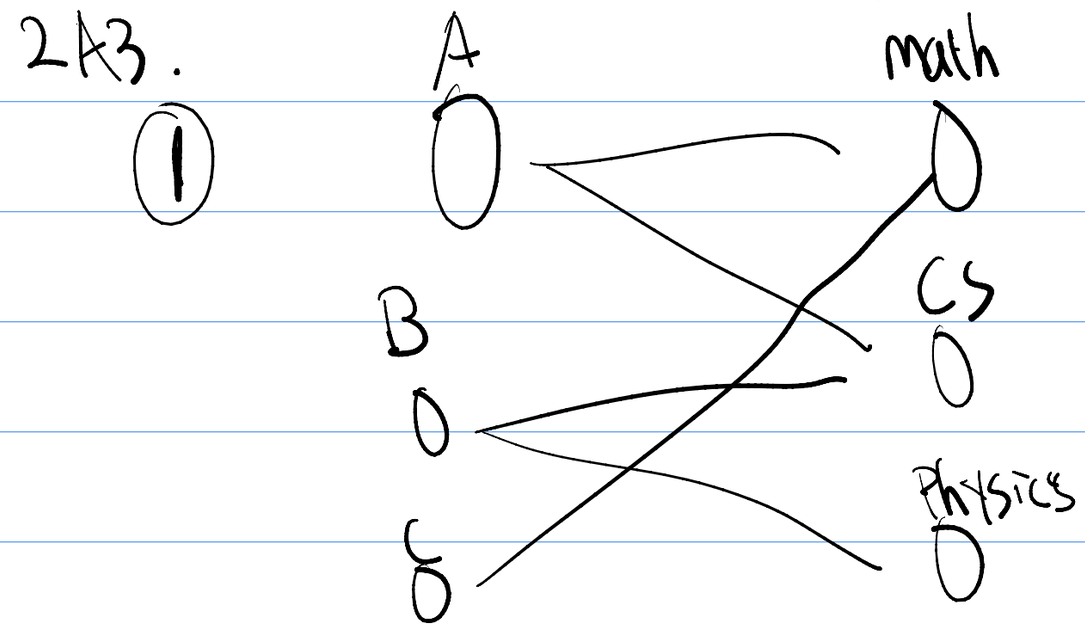
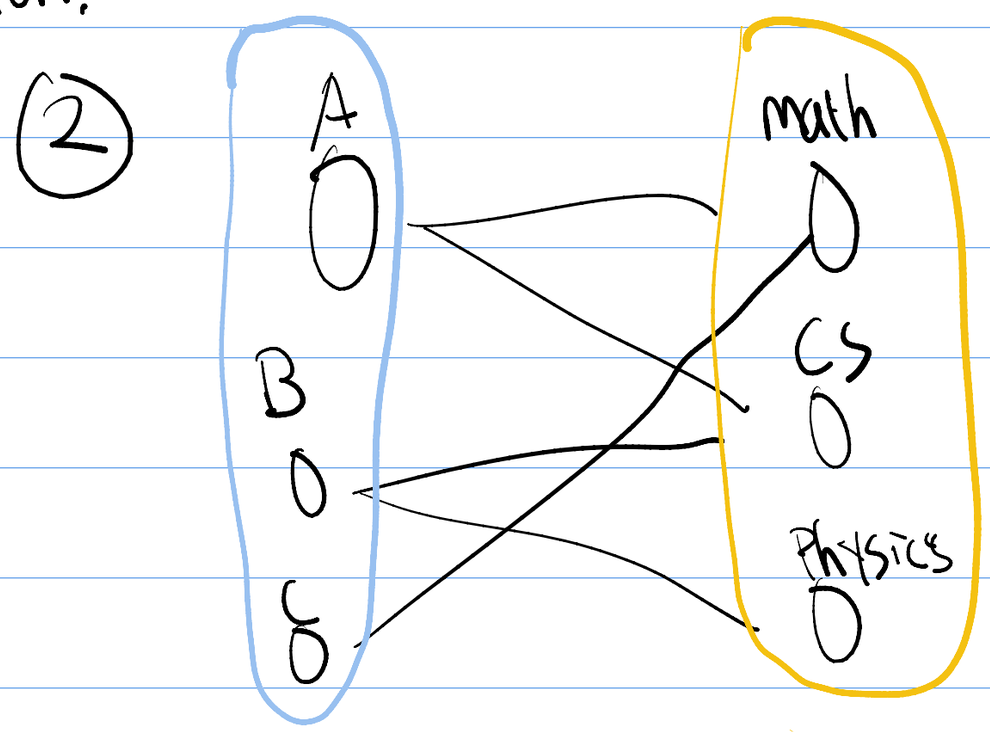
Network ex-02

2A1 Nodes: People within the hospital (patient and staff)
Edges: direct contact
Direction: Irrelevant
Weight: How long does the contact last / how frequent does the contact happens
Does it change: Yes because its based on the SIR model so nodes can became sick and recover
what does it reveal: which nodes can infects others and need to get treatment
what does it hide: maybe environmental infections

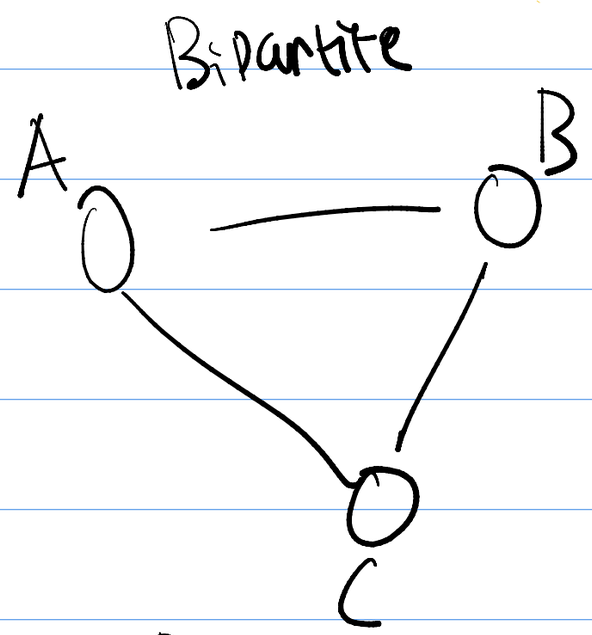
2A2 1. A→B in model R means A (router) would connect to each other B (router) by connecting the LAN cable.
2. A sequence of institutions reachable from each other. this will hide router level details, one hop in model U may represent dozens of cable connections
3. MIT to Stanford uses model U, if its regarding institutional connection and not physical cable connection, use model U
4. Use model R if we want to see which cable disconnects the network (bridge) in model U, cables are invisible.
5. Different model based on the use cases. Using wrong model for specific situation will hide relevant information.

2A3.





③ DA lets us analyze whether a course is desirable or not (which course has the most students)
④ whether they are groups / social clusters
⑤ The specific courses connecting the students together



2.B.1

① $V = \{A, B, C, D, E\}$ $E = \{\{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{D, E\}\}$

② Adjacency matrix:

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

③ Adjacency list : {"A": ["B", "C"], "B": ["A", "C", "D"], "C": ["A", "B"], "D": ["B", "E"], "E": ["D"]

④ $V = 5$ $E = 5$

⑤ $\deg(A) = 2$
$\deg(B) = 3$
$\deg(C) = 2$
$\deg(D) = 2$
$\deg(E) = 1$

$10 = 2|E|$
$10 = 2.5$
$10 = 10$ ✓

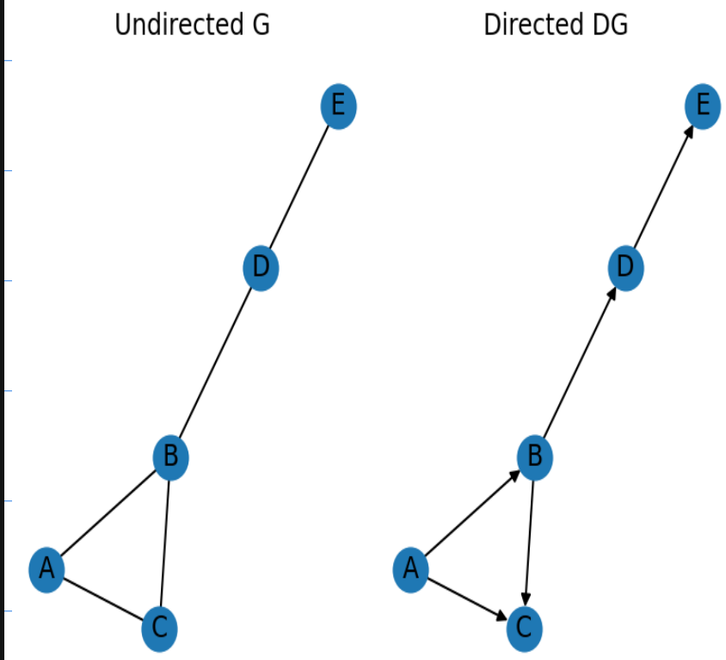
⑥ matrix loses symmetry, B out degree : 3 B in degree = 2

2.B.2

study > snippets > network-ex02 > 2.B.2.py > ...

```python
import matplotlib.pyplot as plt
import networkx as nx
G = nx.Graph()
G.add_edges_from([("A", "B"), ("A", "C"), ("B", "C"), ("B", "D"),("D","E")])
nodes = sorted(G.nodes())

#1
print("Adjacency matrix:\n", nx.to_numpy_array(G, nodelist=nodes))

#2
print("Degrees:", dict(G.degree()))

#3
print(f"Density: {nx.density(G):.3f}")

#4 Directed version
DG = nx.DiGraph()
DG.add_edges_from([("A", "B"), ("A", "C"), ("B", "C"), ("B", "D"), ("D", "E")])
print(f"In-degree of B: {DG.in_degree('B')}, Out-degree of B: {DG.out_degree('B')}")
print(f"In-degree of D: {DG.in_degree('D')}, Out-degree of D: {DG.out_degree('D')}")

#5 Plot both graphs side by side
pos = nx.spring_layout(G, seed=42)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
nx.draw(G, pos, with_labels=True, ax=ax1)
ax1.set_title("Undirected G")
nx.draw(DG, pos, with_labels=True, ax=ax2, arrows=True)
ax2.set_title("Directed DG")
plt.tight_layout()
plt.show()
```

Output ↓



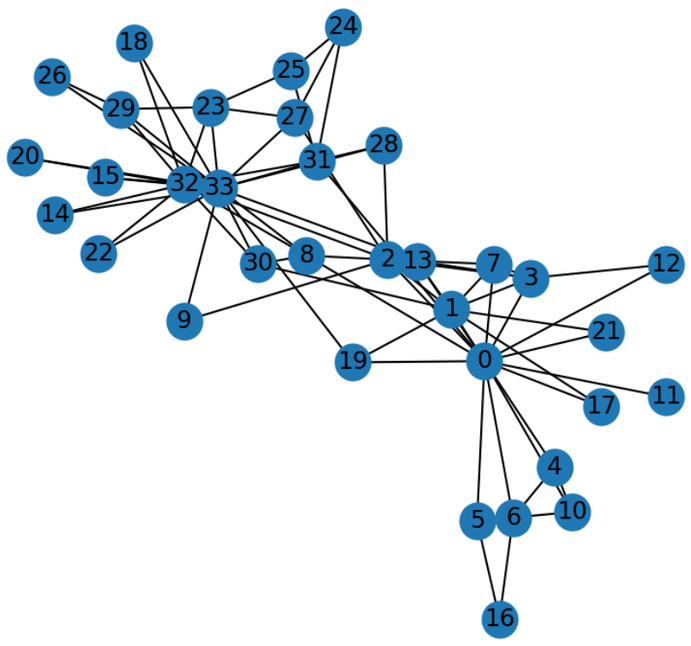

```
❯ python3 2.B.2.py
Adjacency matrix:
 [[0. 1. 1. 0. 0.]
 [1. 0. 1. 1. 0.]
 [1. 1. 0. 0. 0.]
 [0. 1. 0. 0. 1.]
 [0. 0. 0. 1. 0.]]
Degrees: {'A': 2, 'B': 3, 'C': 2, 'D': 2, 'E': 1}
Density: 0.500
In-degree of B: 1, Out-degree of B: 2
In-degree of D: 1, Out-degree of D: 1
```

2.B.3

study > snippets > network-ex02 > 2.B.3.py > ...

```python
import networkx as nx
import matplotlib.pyplot as plt

G=nx.karate_club_graph()
nodes = sorted(G.nodes())
```

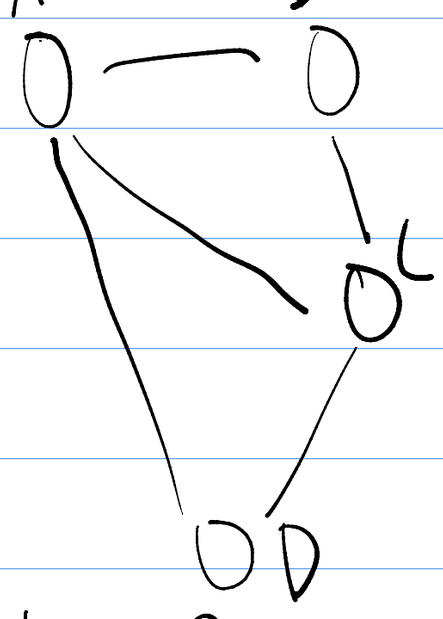
```python
#1
print(f'Number of nodes: {G.number_of_nodes()}, Number of edges: {G.number_of_edges()}')

#2
avg_deg = 2 * G.number_of_edges() / G.number_of_nodes()
print(f'Average degree: {avg_deg:.2f}')

#3
isolated_nodes = [node for node, degree in G.degree() if degree == 0]
print(f'Isolated nodes: {isolated_nodes}')

#4
nx.draw_spring(G, with_labels=True)
plt.show()
```



Result

```
Number of nodes: 34, Number of edges: 78
Average degree: 4.59
Isolated nodes: []
```

2.C.1



ABCA = Cycle
ACDA = Cycle
ABCDA = Cycle
ABCDC = walk (C visited twice)
ABACA = walk (A visited twice)
AE = Doesn't exist

Diameter = 2

2.C.2 ① V = {N, S, I, E} E = {N-I, N-I, S-I, S-I, N-E, S-E, I-E}

② deg(N) = 3
deg(S) = 3
deg(I) = 5
deg(E) = 3

③ Eulerian Circuit = all degree must be even. No eulerian Circuit

④ Eulerian path = requires exactly 2 nodes having odd degree, here's its 4 nodes, So no eulerian path

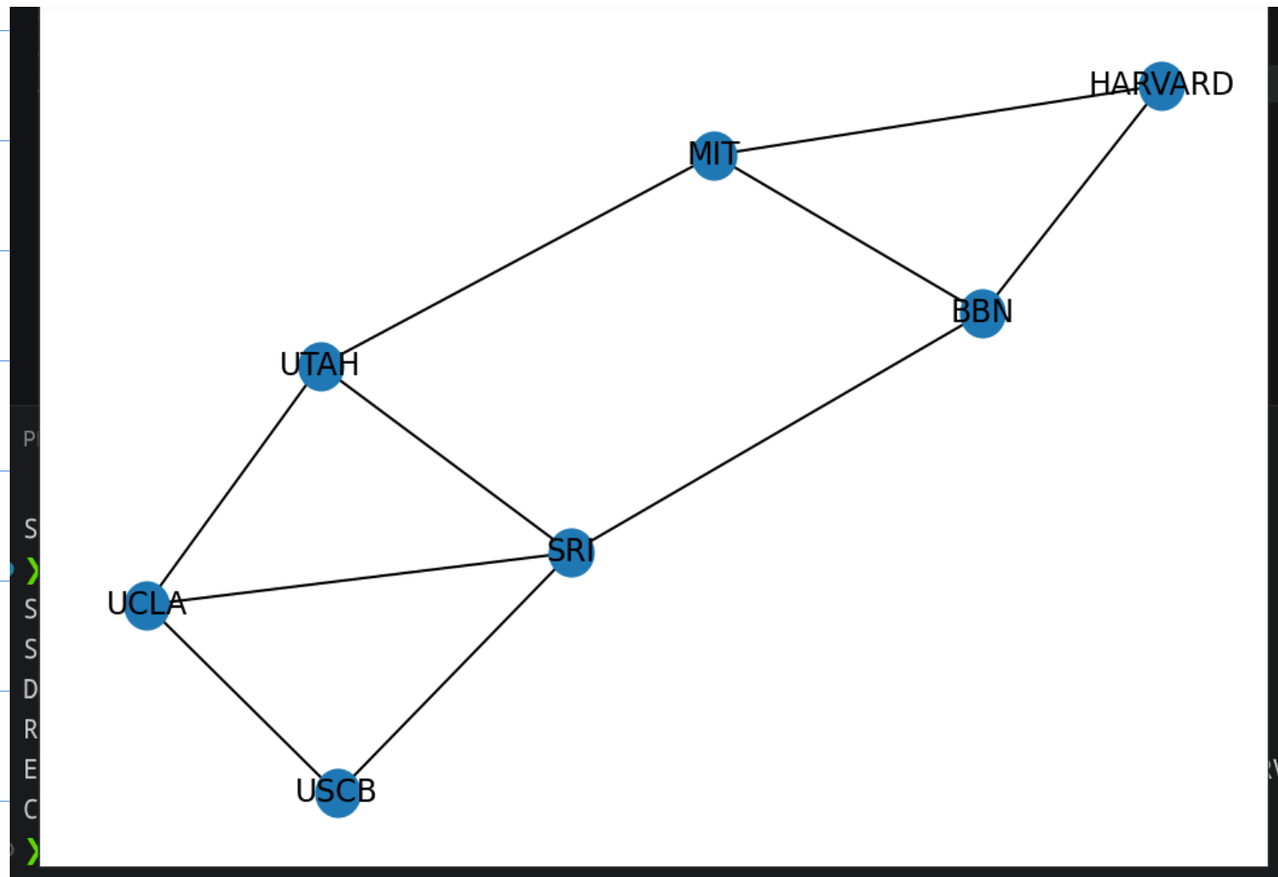
⑤ Euler results tells us only what "connects to what" matters

2.C.3

```python
import networkx as nx
import matplotlib.pyplot as plt

arpa = nx.Graph()
arpa.add_edges_from([
    ("UCLA", "SRI"), ("UCLA", "USCB"), ("UCLA", "UTAH"),
    ("SRI", "UTAH"), ("SRI", "BBN"),
    ("USCB", "SRI"), ("UTAH","MIT"),
```

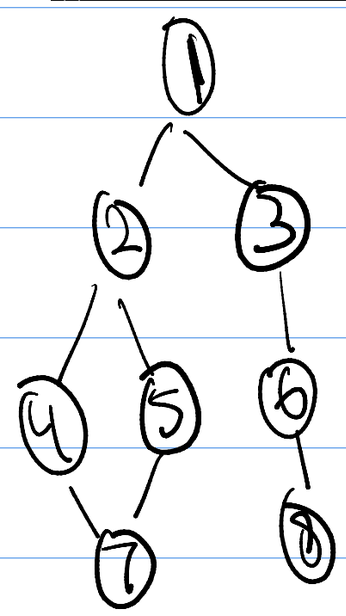
```python
    ("BBN", "HARVARD"), ("BBN", "MIT"), ("MIT", "HARVARD"),
])

#2
path = nx.shortest_path(arpa, "UCLA", "HARVARD")
print(f"Shortest path: {path}")
print(f"Shortest path length: {len(path) - 1}")

#3
print(f"Diameter: {nx.diameter(arpa)}")
print(f"Radius: {nx.radius(arpa)}")
print(f"Eccentricity: {nx.eccentricity(arpa)}")

#4
print(f"Center: {nx.center(arpa)}")

#1
nx.draw_spring(arpa, with_labels=True)
plt.show()
```

Result




```
Shortest path: ['UCLA', 'SRI', 'BBN', 'HARVARD']
Shortest path length: 3
Diameter: 3
Radius: 2
Eccentricity: {'UCLA': 3, 'SRI': 2, 'USCB': 3, 'UTAH': 2, 'BBN': 2, 'MIT': 3, 'HARVARD': 3}
Center: ['SRI', 'UTAH', 'BBN']
```

2D1



Node	layer	parent
1	–	–
2,3	1	1
4,5	2	2
6	2	3
7,8	3	4 or 5; 6

3. $d(1,7) = 3$ and $d(1,8) = 3$, Shortest path to 7 can either be with 4 or 5 as a parent, so more than 1 Shortest path

4. No, BFS count hops not weight. use dijkstra for weighted graphs

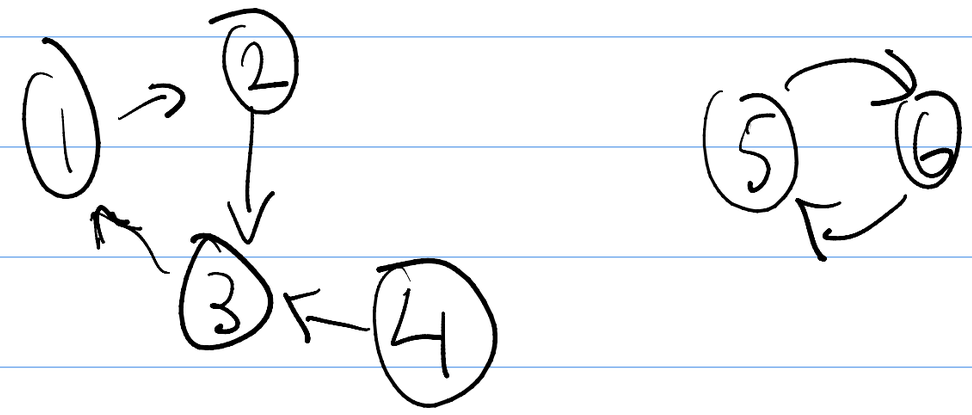
2D2 ③ Ecc of Mr. Hi reaches more nodes in 2 hops than node 33.

```python
from collections import deque
import networkx as nx

#1
def bfs_distances(graph, source):
    dist = {source: 0}
    queue = deque([source])
    while queue:
        node = queue.popleft()
        for neighbor in graph.get(node, []):
            if neighbor not in dist:
                dist[neighbor] = dist[node] + 1
                queue.append(neighbor)
    return dist
G = nx.karate_club_graph()
adj = {node: list(G.neighbors(node)) for node in G.nodes()}

#2 & 4
d0 = bfs_distances(adj, 0)
d33 = bfs_distances(adj, 33)
```

```
print(f"Eccentricity of node 0: {max(d0.values())}, Eccentricity of node 33: {max(d33.values())}")
```

```
{0: 0, 1: 1, 2: 1, 3: 1, 4: 1, 5: 1, 6: 1, 7: 1, 8: 1, 10: 1, 11: 1, 12: 1, 13:
 24: 2, 25: 2, 23: 3, 14: 3, 15: 3, 18: 3, 20: 3, 22: 3, 29: 3, 26: 3}
{0: 0, 1: 1, 2: 1, 3: 1, 4: 1, 5: 1, 6: 1, 7: 1, 8: 1, 10: 1, 11: 1, 12: 1, 13:
 24: 2, 25: 2, 23: 3, 14: 3, 15: 3, 18: 3, 20: 3, 22: 3, 29: 3, 26: 3}
Matches: True
Eccentricity: 3
Eccentricity (NetworkX): 3
❯ python3 2.D.2.py
Eccentricity of node 0: 3, Eccentricity of node 33: 4
❯ python3 2.D.2.py
Eccentricity of node 0: 3, Eccentricity of node 33: 4
❯ python3 2.D.2.py
Eccentricity of node 0: 3, Eccentricity of node 33: 4
```

2.E.1



① {1,2,3,4} and {5,6}

② {1,2,3}, {5,6}

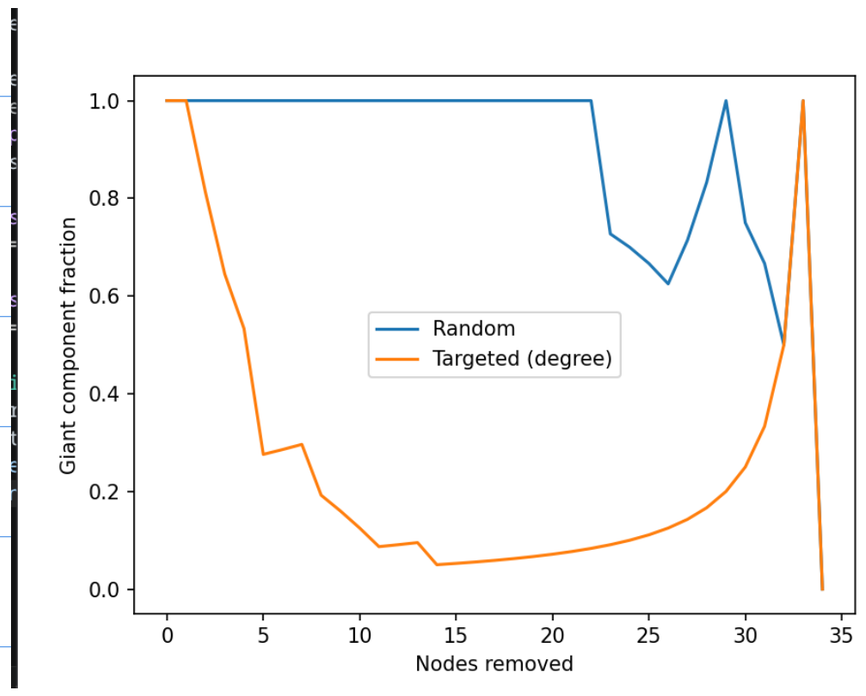
③ if information flows out of node 4 it'll act as a broadcaster information will get distributed but no information will get back

④ cycle breaks, each of 1,2,3 becomes it owns SCC

2.E.2

study > snippets > network-ex02 > 2.E.2.py > ...

```python
import networkx as nx
import matplotlib.pyplot as plt
import random

G = nx.karate_club_graph()

def giant_fraction(G):
    if G.number_of_nodes() == 0:
        return 0
    return len (max(nx.connected_components(G), key=len)) / G.number_of_nodes()

def simulate(G0,strategy) :
    G = G0.copy()
    sizes = [giant_fraction(G)]
    while G.number_of_nodes() > 0:
        if strategy == "random":
            node = random.choice(list(G.nodes()))
        else:
            node = max(G.nodes(), key=lambda n: G.degree(n))
        G.remove_node(node)
        sizes.append(giant_fraction(G))
    return sizes

sizes_random = simulate(G, "random")
sizes_targeted = simulate(G, "targeted")

sizes_random = simulate(G, "random")
sizes_targeted = simulate(G, "targeted")

import matplotlib.pyplot as plt
plt.plot(sizes_random, label="Random")
plt.plot(sizes_targeted, label="Targeted (degree)")
plt.xlabel("Nodes removed")
plt.ylabel("Giant component fraction")
plt.legend()
plt.show()
```



hubs based networks are robust against randomized attacks

but collapses against targeted attacks.